RustHSM FIPS 140-3 Security Policy and Certification Document

Document Information

Field	Value
Module Name	RustHSM
Module Version	0.8.0
Module Type	Software
FIPS 140-3 Overall Security Level	1
PKCS#11 Compliance	v3.0 (OASIS Standard)
Document Version	1.0
Date	2026-03-08
Purpose	FIPS 140-3 Level 1 Certification Submission
Author	RustHSM Project

Revision History

Version	Date	Description
1.0	2026-03-08	Initial certification document

Acronyms and Abbreviations

Acronym	Definition
AES	Advanced Encryption Standard (FIPS 197)
API	Application Programming Interface
CBC	Cipher Block Chaining (SP 800-38A)
CMVP	Cryptographic Module Validation Program
CO	Crypto Officer (PKCS#11 Security Officer role)
CSP	Critical Security Parameter
CTR	Counter mode (SP 800-38A)
DRBG	Deterministic Random Bit Generator
ECDH	Elliptic Curve Diffie-Hellman (SP 800-56A)
ECDSA	Elliptic Curve Digital Signature Algorithm (FIPS 186-5)
GCM	Galois/Counter Mode (SP 800-38D)
HMAC	Hash-based Message Authentication Code (FIPS 198-1)
HSM	Hardware Security Module
KAT	Known Answer Test
KDF	Key Derivation Function
KEM	Key Encapsulation Mechanism
ML-DSA	Module-Lattice Digital Signature Algorithm (FIPS 204)
ML-KEM	Module-Lattice Key Encapsulation Mechanism (FIPS 203)
OAEP	Optimal Asymmetric Encryption Padding
PBKDF2	Password-Based Key Derivation Function 2 (SP 800-132)
PKCS	Public-Key Cryptography Standards
POST	Power-On Self-Test
PQC	Post-Quantum Cryptography
PSS	Probabilistic Signature Scheme
RNG	Random Number Generator
RSA	Rivest-Shamir-Adleman
SLH-DSA	Stateless Hash-based Digital Signature Algorithm (FIPS 205)
SO	Security Officer (PKCS#11 Crypto Officer role)
SP	Special Publication (NIST)
SSP	Sensitive Security Parameter

Normative References

Reference	Title
FIPS 140-3	Security Requirements for Cryptographic Modules
ISO 19790:2012	Security requirements for cryptographic modules
FIPS 180-4	Secure Hash Standard (SHS)
FIPS 186-5	Digital Signature Standard (DSS)
FIPS 197	Advanced Encryption Standard (AES)
FIPS 198-1	The Keyed-Hash Message Authentication Code (HMAC)
FIPS 202	SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions
FIPS 203	Module-Lattice-Based Key-Encapsulation Mechanism Standard
FIPS 204	Module-Lattice-Based Digital Signature Standard
FIPS 205	Stateless Hash-Based Digital Signature Standard
SP 800-38A	Recommendation for Block Cipher Modes of Operation
SP 800-38D	Recommendation for Block Cipher Modes of Operation: GCM
SP 800-38F	Recommendation for Block Cipher Modes of Operation: Key Wrapping
SP 800-56A	Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography
SP 800-57 Part 1	Recommendation for Key Management: General
SP 800-90A Rev.1	Recommendation for Random Number Generation Using Deterministic Random Bit Generators
SP 800-90B	Recommendation for the Entropy Sources Used for Random Bit Generation
SP 800-131A Rev.2	Transitioning the Use of Cryptographic Algorithms and Key Lengths
SP 800-132	Recommendation for Password-Based Key Derivation
PKCS#11 v3.0	OASIS PKCS#11 Cryptographic Token Interface Base Specification

---

1. Cryptographic Module Specification

1.1 Module Identification

Property	Value
Module Name	RustHSM
Version	0.8.0
Type	Software module
Language	Rust (Edition 2021)
Output Artifact	Shared library: `librusthsm.so` (Linux), `rusthsm.dll` (Windows), `librusthsm.dylib` (macOS)
Source	`Cargo.toml` line 5: `name = "rusthsm"`

1.2 Security Level Claim

FIPS 140-3 Area	Security Level
1. Cryptographic Module Specification	1
2. Cryptographic Module Interfaces	1
3. Roles, Services, and Authentication	1 (identity-based)
4. Software/Firmware Security	1
5. Operational Environment	1 (modifiable)
6. Physical Security	N/A (software module)
7. Non-Invasive Security	N/A
8. Sensitive Security Parameter Management	1
9. Self-Tests	1
10. Life-Cycle Assurance	1
11. Mitigation of Other Attacks	N/A
Overall	**1**

1.3 Module Description

RustHSM is a PKCS#11 v3.0-compliant software cryptographic module implemented entirely in the Rust programming language. The module provides cryptographic key management, digital signature, encryption, hashing, key derivation, and key wrapping services through the industry-standard PKCS#11 C API.

The module is compiled as a dynamically loadable shared library (`cdylib`) that applications load at runtime via `dlopen` (Unix) or `LoadLibrary` (Windows). All cryptographic operations, key storage, authentication, and self-tests are contained within the single shared library binary.

The module supports 41 cryptographic mechanisms including post-quantum algorithms (ML-DSA, ML-KEM, SLH-DSA) and offers dual cryptographic backends: RustCrypto (default, pure Rust) and aws-lc-rs (optional, FIPS 140-3 validated).

1.4 Module Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                    CRYPTOGRAPHIC MODULE BOUNDARY                 │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │              PKCS#11 C ABI Layer (68+ functions)          │    │
│  │              src/pkcs11_abi/functions.rs                   │    │
│  │       Input validation, catch_unwind, CK_RV mapping       │    │
│  └──────────┬──────────┬──────────┬──────────┬──────────────┘    │
│             │          │          │          │                   │
│  ┌──────────▼──┐ ┌─────▼──────┐ ┌▼─────────┐ ┌▼──────────────┐   │
│  │ SessionMgr│ │ ObjectStore│ │ AuditLog │ │  CryptoBackend│   │
│  │ DashMap   │ │ DashMap +  │ │ chained  │ │  trait (26    │   │
│  │ sessions  │ │ redb+AES   │ │ SHA-256  │ │  methods)     │   │
│  └───────────┘ └────────────┘ └──────────┘ └──────┬────────┘   │
│                      │                            │             │
│              ┌───────▼────────┐           ┌────────▼─────────┐  │
│              │ RawKeyMaterial│           │  RustCryptoBackend │  │
│              │ mlock+zeroize │           │  --- OR ---        │  │
│              └────────────────┘           │  AwsLcBackend      │  │
│                                           │  (FIPS validated)  │  │
│  ┌──────────────────┐                     └────────────────────┘  │
│  │  HMAC_DRBG       │  ┌──────────────────┐                     │
│  │  SP 800-90A      │  │  POST/KAT Engine │                     │
│  │  (prediction     │  │  15 KATs +       │                     │
│  │   resistance)    │  │  integrity test   │                     │
│  └──────────────────┘  └──────────────────┘                     │
│                                                                  │
│                        HsmCore (src/core.rs)                     │
│                   OnceLock<Arc<HsmCore>> singleton               │
└──────────────────────────────────────────────────────────────────┘
```

1.5 Module Boundary

Components inside the cryptographic boundary:

Component	Source Path	Description
PKCS#11 C ABI	`src/pkcs11_abi/`	68+ exported C functions, types, constants
Cryptographic algorithms	`src/crypto/`	Sign, encrypt, digest, keygen, PQC, DRBG
Self-tests	`src/crypto/self_test.rs`	15 POST KATs + software integrity test
Pairwise consistency	`src/crypto/pairwise_test.rs`	7 conditional self-tests
Key material protection	`src/store/key_material.rs`	RawKeyMaterial with mlock + zeroize
Object storage	`src/store/`	ObjectStore, EncryptedStore (redb + AES-256-GCM)
Session management	`src/session/`	SessionManager, Session state machine
Token/PIN management	`src/token/`	Token initialization, PIN hashing, login state
Audit logging	`src/audit/`	Tamper-evident append-only log
Module core	`src/core.rs`	HsmCore coordinator struct
Error mapping	`src/error.rs`	HsmError enum to CK_RV conversion

Components outside the cryptographic boundary:

Component	Path	Description
gRPC daemon	`rusthsm-daemon/`	Network transport layer
Admin CLI	`tools/rusthsm-admin/`	Command-line token management
Spy wrapper	`tools/pkcs11-spy/`	Debug/logging interceptor
Deployment	`deploy/`	Dockerfile, Helm chart
Third-party crates	`Cargo.lock`	Evaluated separately for FIPS compliance

1.6 Modes of Operation

The module operates in one of two modes:

FIPS Approved Mode (configuration: `fips_approved_only = true`, `crypto_backend = "awslc"`):

- Only FIPS-approved algorithms are available
- Non-approved mechanisms return `CKR_MECHANISM_INVALID`
- PQC mechanisms are blocked
- Ed25519 is blocked
- SHA-1 for signing is blocked
- All operations are performed through the FIPS-validated aws-lc-rs backend
- Source: `src/crypto/mechanisms.rs:165` (`is_fips_approved()`)

Non-Approved Mode (default configuration):

- All 41 mechanisms are available including PQC and Ed25519
- Algorithm indicator (IG 2.4.C) marks each operation as approved or non-approved
- Uses the RustCrypto backend (not FIPS-validated)

---

2. Cryptographic Module Interfaces

2.1 Logical Interface Mapping

Per FIPS 140-3 §7.2, all module interfaces are mapped to the four logical interface types:

FIPS 140-3 Interface	PKCS#11 Mapping	Direction
Data Input	Plaintext data in `C_Encrypt`, `C_Sign`, `C_Digest`, `C_DigestUpdate`; key material in `C_CreateObject`, `C_UnwrapKey` templates; PIN values in `C_Login`, `C_InitPIN`, `C_SetPIN`	Input
Data Output	Ciphertext from `C_Encrypt`; signatures from `C_Sign`; digests from `C_Digest`; key material from `C_GetAttributeValue`; wrapped keys from `C_WrapKey`; random bytes from `C_GenerateRandom`	Output
Control Input	`C_Initialize`, `C_Finalize`, `C_InitToken`, `C_Login`, `C_Logout`, `C_OpenSession`, `C_CloseSession`, mechanism selection in `*Init` functions, template attributes in `C_GenerateKey` / `C_GenerateKeyPair`	Input
Status Output	`CK_RV` return codes from all functions; `CK_INFO` from `C_GetInfo`; `CK_SLOT_INFO` from `C_GetSlotInfo`; `CK_TOKEN_INFO` from `C_GetTokenInfo`; `CK_SESSION_INFO` from `C_GetSessionInfo`; `CK_MECHANISM_INFO` from `C_GetMechanismInfo`	Output

2.2 Exported API Functions

The module exports the following 68 C functions through `C_GetFunctionList`. All functions are `#[no_mangle] pub extern "C"` with `catch_unwind` to prevent panic propagation across the FFI boundary.

Source: `src/pkcs11_abi/functions.rs`

General-Purpose Functions:

Function	Interface	Description
`C_Initialize`	Control	Initialize the module, run POST
`C_Finalize`	Control	Shut down the module
`C_GetInfo`	Status	Return library info (version, manufacturer)
`C_GetFunctionList`	Status	Return function pointer table

Slot and Token Management:

Function	Interface	Description
`C_GetSlotList`	Status	Enumerate available slots
`C_GetSlotInfo`	Status	Return slot description
`C_GetTokenInfo`	Status	Return token status and capabilities
`C_GetMechanismList`	Status	Enumerate supported mechanisms
`C_GetMechanismInfo`	Status	Return mechanism capabilities
`C_InitToken`	Control	Initialize/reinitialize the token (CO role)
`C_InitPIN`	Control	Set user PIN (CO role)
`C_SetPIN`	Control	Change PIN (CO or User role)

Session Management:

Function	Interface	Description
`C_OpenSession`	Control	Open a new session (RO or RW)
`C_CloseSession`	Control	Close a session
`C_CloseAllSessions`	Control	Close all sessions on a slot
`C_GetSessionInfo`	Status	Return session state
`C_Login`	Control	Authenticate as CO or User
`C_Logout`	Control	End authenticated session

Object Management:

Function	Interface	Description
`C_CreateObject`	Control/Data	Import a key or data object
`C_CopyObject`	Control	Duplicate an object
`C_DestroyObject`	Control	Destroy an object (zeroize key material)
`C_GetObjectSize`	Status	Return approximate object size
`C_GetAttributeValue`	Data Output	Read object attributes
`C_SetAttributeValue`	Control	Modify object attributes
`C_FindObjectsInit`	Control	Begin object search
`C_FindObjects`	Data Output	Return matching object handles
`C_FindObjectsFinal`	Control	End object search

Encryption/Decryption:

Function	Interface	Description
`C_EncryptInit`	Control	Initialize encryption operation
`C_Encrypt`	Data I/O	Single-part encryption
`C_EncryptUpdate`	Data I/O	Multi-part encryption
`C_EncryptFinal`	Data Output	Finalize multi-part encryption
`C_DecryptInit`	Control	Initialize decryption operation
`C_Decrypt`	Data I/O	Single-part decryption
`C_DecryptUpdate`	Data I/O	Multi-part decryption
`C_DecryptFinal`	Data Output	Finalize multi-part decryption

Signing/Verification:

Function	Interface	Description
`C_SignInit`	Control	Initialize signing operation
`C_Sign`	Data I/O	Single-part signing
`C_SignUpdate`	Data Input	Multi-part signing
`C_SignFinal`	Data Output	Finalize multi-part signing
`C_VerifyInit`	Control	Initialize verification operation
`C_Verify`	Data I/O	Single-part verification
`C_VerifyUpdate`	Data Input	Multi-part verification
`C_VerifyFinal`	Status	Finalize multi-part verification

Message Digest:

Function	Interface	Description
`C_DigestInit`	Control	Initialize digest operation
`C_Digest`	Data I/O	Single-part digest
`C_DigestUpdate`	Data Input	Multi-part digest
`C_DigestKey`	Data Input	Hash key material into digest
`C_DigestFinal`	Data Output	Finalize multi-part digest

Key Management:

Function	Interface	Description
`C_GenerateKey`	Control	Generate symmetric key
`C_GenerateKeyPair`	Control	Generate asymmetric key pair
`C_WrapKey`	Data Output	Export key in wrapped form
`C_UnwrapKey`	Data Input	Import wrapped key
`C_DeriveKey`	Control	Derive key from base key

Random Number Generation:

Function	Interface	Description
`C_SeedRandom`	Control	Add entropy (returns `CKR_RANDOM_SEED_NOT_SUPPORTED`)
`C_GenerateRandom`	Data Output	Generate random bytes via HMAC_DRBG

Not Implemented (return `CKR_FUNCTION_NOT_SUPPORTED`):

`C_GetOperationState`, `C_SetOperationState`, `C_SignRecoverInit`, `C_SignRecover`, `C_VerifyRecoverInit`, `C_VerifyRecover`, `C_DigestEncryptUpdate`, `C_DecryptDigestUpdate`, `C_SignEncryptUpdate`, `C_DecryptVerifyUpdate`, `C_GetFunctionStatus`, `C_CancelFunction`, `C_WaitForSlotEvent`

2.3 Interface Separation

The module maintains strict separation between interface types:

- Control inputs (PINs, mechanism selections) are never emitted as data outputs
- Status outputs (CK_RV codes) do not leak internal state — error codes are generic per PKCS#11 specification
- Data outputs (ciphertext, signatures, digests) flow only through designated output parameters
- The error mapping at `src/error.rs:104-157` converts internal `HsmError` variants to standardized `CK_RV` codes, preventing information leakage through error specificity

---

3. Roles, Services, and Authentication

3.1 Roles

The module implements three roles per the PKCS#11 specification:

Role	PKCS#11 User Type	Description	Authentication
Crypto Officer (CO)	`CKU_SO` (0)	Token initialization, user PIN management	PIN-based (PBKDF2-HMAC-SHA256)
User	`CKU_USER` (1)	Cryptographic operations, object management	PIN-based (PBKDF2-HMAC-SHA256)
Unauthenticated	(none)	Limited services: digest, random, public info	None

Source: `src/session/session.rs:7-13` (SessionState enum), `src/token/token.rs` (login handling)

3.2 Services-to-Roles Mapping

The following table maps all module services to the roles authorized to access them, the CSPs accessed, and the FIPS approval status.

Token Management Services:

Service	API Function	CO	User	Unauth	CSP Access
Initialize token	`C_InitToken`	Yes	No	No	SO PIN hash (output)
Initialize user PIN	`C_InitPIN`	Yes	No	No	User PIN hash (output)
Change own PIN	`C_SetPIN`	Yes	Yes	No	PIN hash (input/output)
Login	`C_Login`	Yes	Yes	No	PIN hash (input)
Logout	`C_Logout`	Yes	Yes	No	None

Symmetric Key Generation (Approved):

Service	Mechanism	CO	User	Unauth	CSP Access
AES key generation	`CKM_AES_KEY_GEN`	No	Yes	No	AES key (output), DRBG state (input)

Asymmetric Key Pair Generation:

Service	Mechanism	CO	User	Unauth	FIPS Approved	CSP Access
RSA keygen	`CKM_RSA_PKCS_KEY_PAIR_GEN`	No	Yes	No	Yes	RSA private key (output), DRBG state
EC P-256/P-384 keygen	`CKM_EC_KEY_PAIR_GEN`	No	Yes	No	Yes	EC private key (output), DRBG state
Ed25519 keygen	`CKM_EDDSA`	No	Yes	No	No	Ed25519 key (output), DRBG state
ML-DSA keygen	`CKM_ML_DSA_44/65/87`	No	Yes	No	No	ML-DSA key (output), DRBG state
ML-KEM keygen	`CKM_ML_KEM_512/768/1024`	No	Yes	No	No	ML-KEM key (output), DRBG state
SLH-DSA keygen	`CKM_SLH_DSA_SHA2_128S/256S`	No	Yes	No	No	SLH-DSA key (output), DRBG state

Digital Signature Services:

Service	Mechanism	FIPS Approved	CSP Access
RSA PKCS#1 v1.5 sign	`CKM_RSA_PKCS`	Yes	RSA private key (input)
RSA PKCS#1 v1.5 sign (hashed)	`CKM_SHA256/384/512_RSA_PKCS`	Yes	RSA private key (input)
RSA-PSS sign	`CKM_RSA_PKCS_PSS`	Yes	RSA private key (input)
RSA-PSS sign (hashed)	`CKM_SHA256/384/512_RSA_PKCS_PSS`	Yes	RSA private key (input)
ECDSA sign	`CKM_ECDSA`	Yes	EC private key (input)
ECDSA sign (hashed)	`CKM_ECDSA_SHA256/384/512`	Yes	EC private key (input)
Ed25519 sign	`CKM_EDDSA`	No	Ed25519 private key (input)
ML-DSA sign	`CKM_ML_DSA_44/65/87`	No	ML-DSA private key (input)
SLH-DSA sign	`CKM_SLH_DSA_SHA2_128S/256S`	No	SLH-DSA private key (input)

All signing services require User authentication. Corresponding verify services use public key components and also require User authentication.

Encryption/Decryption Services:

Service	Mechanism	FIPS Approved	CSP Access
AES-GCM encrypt/decrypt	`CKM_AES_GCM`	Yes	AES key (input)
AES-CBC encrypt/decrypt	`CKM_AES_CBC` / `CKM_AES_CBC_PAD`	Yes	AES key (input)
AES-CTR encrypt/decrypt	`CKM_AES_CTR`	Yes	AES key (input)
RSA-OAEP encrypt/decrypt	`CKM_RSA_PKCS_OAEP`	Yes	RSA key (input)

All encryption/decryption services require User authentication.

Key Wrapping/Unwrapping Services:

Service	Mechanism	FIPS Approved	CSP Access
AES Key Wrap	`CKM_AES_KEY_WRAP`	Yes	Wrapping key (input), target key (input)
AES Key Wrap with Padding	`CKM_AES_KEY_WRAP_KWP`	Yes	Wrapping key (input), target key (input)

Key Derivation Services:

Service	Mechanism	FIPS Approved	CSP Access
ECDH P-256 derive	`CKM_ECDH1_DERIVE`	Yes	EC private key (input), derived key (output)
ECDH P-384 derive	`CKM_ECDH1_DERIVE`	Yes	EC private key (input), derived key (output)
ECDH cofactor derive	`CKM_ECDH1_COFACTOR_DERIVE`	Yes	EC private key (input), derived key (output)

Digest Services (No CSP access):

Service	Mechanism	FIPS Approved
SHA-1 digest	`CKM_SHA_1`	Yes (digest only, not for signing)
SHA-256 digest	`CKM_SHA256`	Yes
SHA-384 digest	`CKM_SHA384`	Yes
SHA-512 digest	`CKM_SHA512`	Yes
SHA3-256 digest	`CKM_SHA3_256`	Yes
SHA3-384 digest	`CKM_SHA3_384`	Yes
SHA3-512 digest	`CKM_SHA3_512`	Yes

Digest services are available to all roles including unauthenticated.

Random Number Generation:

Service	API Function	FIPS Approved	CSP Access
Generate random	`C_GenerateRandom`	Yes	DRBG state (input/output)

Available to User and unauthenticated roles.

Object Management Services:

Service	API Function	CO	User	Unauth
Create object	`C_CreateObject`	Yes (RW)	Yes (RW)	No
Copy object	`C_CopyObject`	Yes (RW)	Yes (RW)	No
Destroy object	`C_DestroyObject`	Yes (RW)	Yes (RW)	No
Get attributes	`C_GetAttributeValue`	Yes	Yes	Public objects only
Set attributes	`C_SetAttributeValue`	Yes (RW)	Yes (RW)	No
Find objects	`C_FindObjects*`	Yes	Yes	Public objects only
Get object size	`C_GetObjectSize`	Yes	Yes	No

3.3 Authentication Mechanism

Algorithm: PBKDF2-HMAC-SHA256 (SP 800-132)

Parameter	Value	Source
PRF	HMAC-SHA256	`src/store/encrypted_store.rs`
Iteration count	600,000	Configurable via `security.pbkdf2_iterations`
Salt length	32 bytes (random, per-token)	Generated via `OsRng`
Derived key length	32 bytes	Used for PIN hash comparison
PIN length	4-64 bytes	Configurable via `security.pin_min_length`, `security.pin_max_length`

Comparison: Constant-time via `subtle::ConstantTimeEq` to prevent timing side-channel attacks.

Brute-Force Protection:

Parameter	Value
Max failed SO login attempts	10 (configurable)
Max failed User login attempts	10 (configurable)
Lockout behavior	Returns `CKR_PIN_LOCKED` (0xA4)
Lockout recovery	SO resets User PIN via `C_InitPIN`

Source: `src/token/token.rs` (login state, failed attempt counters)

3.4 Authentication Strength

With a minimum 4-character PIN (assuming alphanumeric, 62 possible characters):

- PIN space: 62^4 = 14,776,336 possible PINs
- PBKDF2 cost: 600,000 iterations per guess
- Online brute-force cost: 14,776,336 × 600,000 = ~8.9 × 10^12 HMAC-SHA256 operations
- Online defense: 10-attempt lockout makes online brute-force infeasible

With the recommended 8-character PIN:

- PIN space: 62^8 = 2.18 × 10^14 possible PINs

---

4. Software/Firmware Security

4.1 Approved Integrity Technique

The module implements a software integrity test per FIPS 140-3 §9.4 using HMAC-SHA256.

Implementation: `src/crypto/integrity.rs`

Component	Value
Algorithm	HMAC-SHA256 (FIPS 198-1)
HMAC key	32-byte compile-time constant embedded in binary (per FIPS IG 9.7)
Key value	`b"RustHSM-FIPS-Integrity-Key-v1.0!"` (`integrity.rs:23`)
Expected digest	Stored in `.hmac` sidecar file (64 hex characters)
Execution	First test in `run_post()`, before any algorithm KATs

Module Path Discovery:

- **Unix:** `dladdr()` on function pointer address (`integrity.rs:141-156`)
- **Windows:** `GetModuleHandleExW()` + `GetModuleFileNameW()` (`integrity.rs:159-197`)

Behavior Matrix:

Condition	Behavior
No `.hmac` sidecar file	Warning logged, test passes (development mode)
`.hmac` present, HMAC matches	Test passes, module proceeds to KATs
`.hmac` present, HMAC mismatch	Test fails, `POST_FAILED` set, module enters error state
Module path cannot be determined	Warning logged, test passes

Integrity Tool: `tools/compute-integrity-hmac.sh` and `.ps1` compute the `.hmac` sidecar file from the compiled binary for production deployment.

4.2 Executable Code Integrity

- The module contains no dynamically loaded code at runtime
- No self-modifying code is present

- All cryptographic functions are statically compiled into the shared library
- The `crate-type = ["cdylib", "rlib"]` configuration (`Cargo.toml:12`) produces a single artifact

---

5. Operational Environment

5.1 Operating Environment Type

The module operates in a **modifiable operational environment** (general-purpose operating system). Per FIPS 140-3, a single operator is assumed per instantiation of the module.

5.2 Tested Platforms

Operating System	Architecture	Rust Target Triple	Status
Windows 11 Pro 10.0.26200	x86_64	`x86_64-pc-windows-msvc`	Primary development/test platform
Linux (glibc 2.31+)	x86_64	`x86_64-unknown-linux-gnu`	CI tested
Linux (musl)	x86_64	`x86_64-unknown-linux-musl`	Expected compatible
macOS (Intel)	x86_64	`x86_64-apple-darwin`	Expected compatible
macOS (Apple Silicon)	aarch64	`aarch64-apple-darwin`	Expected compatible

5.3 Platform-Specific Security Features

Feature	Linux	Windows	macOS
Memory locking	`mlock(2)` / `munlock(2)`	`VirtualLock` / `VirtualUnlock`	`mlock(2)` / `munlock(2)`
Fork detection	PID comparison via `getpid()`	N/A (CreateProcess is isolated)	PID comparison via `getpid()`
File locking	`flock(2)` via fs2 crate	`LockFileEx` via fs2 crate	`flock(2)` via fs2 crate
Entropy source	`getrandom(2)`	`BCryptGenRandom`	`SecRandomCopyBytes`

5.4 Build Requirements for FIPS Mode

Dependency	Version	Purpose
Rust toolchain	1.75+	Compilation
CMake	3.x+	aws-lc-rs build system
Clang/LLVM	Latest	aws-lc-rs C compilation
Go	1.18+	aws-lc-rs build tool

---

6. Physical Security

Not applicable. RustHSM is a software cryptographic module. The cryptographic boundary is the process address space of the running application. FIPS 140-3 Level 1 does not impose physical security requirements on software modules.

---

7. Non-Invasive Security

Not applicable. FIPS 140-3 Level 1 does not require non-invasive security mechanisms.

---

8. Sensitive Security Parameter Management

8.1 CSP Inventory

The following table enumerates all Critical Security Parameters (CSPs) managed by the module:

#	CSP	Type	Size	Storage	Generation	Zeroization
1	AES-256 symmetric key	Symmetric	32 bytes	`RawKeyMaterial`	HMAC_DRBG (SP 800-90A)	`zeroize()` then `munlock()`
2	AES-192 symmetric key	Symmetric	24 bytes	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
3	AES-128 symmetric key	Symmetric	16 bytes	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
4	RSA private key	Asymmetric	Variable (PKCS#8 DER)	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
5	ECDSA P-256 private key	Asymmetric	32 bytes	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
6	ECDSA P-384 private key	Asymmetric	48 bytes	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
7	Ed25519 private key	Asymmetric	32 bytes	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
8	ML-DSA signing key	PQC	Variable	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
9	ML-KEM decapsulation key	PQC	Variable	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
10	SLH-DSA signing key	PQC	Variable	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
11	HMAC key	Symmetric	Variable	`RawKeyMaterial`	HMAC_DRBG	`zeroize()` then `munlock()`
12	SO PIN hash	Authentication	64 bytes (32 salt + 32 hash)	`Token` struct (`Zeroizing<Vec<u8>>`)	PBKDF2-HMAC-SHA256	`Zeroizing` drop
13	User PIN hash	Authentication	64 bytes (32 salt + 32 hash)	`Token` struct (`Zeroizing<Vec<u8>>`)	PBKDF2-HMAC-SHA256	`Zeroizing` drop
14	Storage encryption key	Symmetric	32 bytes	`ObjectStore` (Mutex)	PBKDF2 from SO PIN	Cleared on logout/reinit
15	HMAC_DRBG K (key)	DRBG internal	32 bytes	`HmacDrbg` struct	SP 800-90A instantiate	`#[zeroize(drop)]`
16	HMAC_DRBG V (value)	DRBG internal	32 bytes	`HmacDrbg` struct	SP 800-90A instantiate	`#[zeroize(drop)]`
17	DRBG last_output	Health test	32 bytes	`HmacDrbg` struct	DRBG generate	`#[zeroize(drop)]`
18	Integrity HMAC key	Compile-time	32 bytes	Binary constant	Compile-time	N/A (constant, not secret)
19	Intermediate plaintext	Transient	Variable	`ActiveOperation.data`	During multi-part ops	`Zeroizing<Vec<u8>>` drop
20	Mechanism parameters	Transient	Variable	`ActiveOperation.mechanism_param`	During operation init	`Zeroizing<Vec<u8>>` drop
21	ECDH shared secret	Derived	32 or 48 bytes	`RawKeyMaterial`	ECDH derivation	`zeroize()` then `munlock()`
22	AES-GCM nonces	Ephemeral	12 bytes	Stack	`OsRng`	Stack deallocation
23	PBKDF2 salt	Public parameter	32 bytes	Stored alongside PIN hash	`OsRng`	N/A (not secret)

Source: CSPs #1-11 stored in `src/store/key_material.rs:10` (`RawKeyMaterial`); #12-13 in `src/token/token.rs` ; #15-17 in `src/crypto/drbg.rs:33-44` ; #19-20 in `src/session/session.rs:39-71`

8.2 CSP Generation

All cryptographic keys are generated using the module's HMAC_DRBG (SP 800-90A Rev.1):

DRBG Property	Value	Source
Algorithm	HMAC_DRBG with HMAC-SHA256	`src/crypto/drbg.rs:22`
Entropy source	OS CSPRNG (`OsRng` → `getrandom(2)` / `BCryptGenRandom`)	`drbg.rs:76`
Seed material	32 bytes entropy + 16 bytes nonce	`drbg.rs:74-78`
Prediction resistance	Enabled (reseeds from OS on every `generate()`)	`drbg.rs:29-31`
Reseed interval	2^48 (per SP 800-90A Table 2)	`drbg.rs:25`
Continuous health test	Consecutive output comparison (SP 800-90B §4.9)	`drbg.rs:42-44`
State zeroization	`#[derive(ZeroizeOnDrop)]` on K, V, last_output	`drbg.rs:32-36`

PIN hashes are generated using PBKDF2-HMAC-SHA256 with 600,000 iterations and a 32-byte random salt.

8.3 CSP Entry and Establishment

Entry Method	API	CSPs Affected
Key import	`C_CreateObject` with `CKA_VALUE`	All key types (#1-11)
Key unwrapping	`C_UnwrapKey` with `CKM_AES_KEY_WRAP`	Symmetric keys (#1-3, 11)
PIN entry	`C_Login`, `C_InitToken`, `C_InitPIN`, `C_SetPIN`	PIN hashes (#12-13)
DRBG seeding	`HmacDrbg::new()` at `C_Initialize`	DRBG state (#15-17)
Key derivation	`C_DeriveKey` with `CKM_ECDH1_DERIVE`	ECDH shared secret (#21)

PIN values are transmitted as plaintext over the PKCS#11 API and immediately hashed via PBKDF2. The plaintext PIN is not stored.

8.4 CSP Output

Output Method	API	CSPs Affected	Conditions
Key export	`C_GetAttributeValue(CKA_VALUE)`	Keys #1-11	`CKA_EXTRACTABLE = true` AND `CKA_SENSITIVE = false`
Key wrapping	`C_WrapKey`	Keys #1-3, 11	`CKA_EXTRACTABLE = true` on target key
Random bytes	`C_GenerateRandom`	DRBG state #15-17 (consumed, not output)	Always available

The following CSPs are **never output** through any API:

- PIN hashes (#12-13)
- Storage encryption key (#14)
- DRBG internal state (#15-17)
- Integrity HMAC key (#18)
- Intermediate plaintext (#19-20)

8.5 CSP Storage

In-Memory Storage:

All key material is stored in `RawKeyMaterial` (`src/store/key_material.rs:10`), which provides:

1. **Memory locking on allocation:** `mlock()` (Unix) or `VirtualLock()` (Windows) prevents the OS from paging key material to swap/disk. Source: `key_material.rs:14-18`
2. **Zeroization on deallocation:** `self.0.zeroize()` overwrites all bytes with 0x00 before `munlock()`. Source: `key_material.rs:42-50`
3. **Debug redaction:** Custom `Debug` impl displays `[REDACTED]` instead of key bytes. Source: `key_material.rs:54-61`
4. **Clone re-locking:** Cloned key material is independently `mlock`'d. Source: `key_material.rs:35-39`

Persistent Storage (Optional):

When configured with `storage_path`, objects are persisted using:

- **Database:** redb (embedded key-value store)
- **Encryption:** Per-object AES-256-GCM encryption
- **Key derivation:** Storage key derived from SO PIN via PBKDF2
- **File locking:** `fs2::FileExt::try_lock_exclusive()` prevents concurrent access
- Source: `src/store/encrypted_store.rs`

8.6 CSP Zeroization

All CSPs are zeroized when no longer needed. The zeroization lifecycle for key material:

```
RawKeyMaterial::new(data)
    │
    ├── mlock_buffer(ptr, len)          ← Lock pages in physical memory
    │
    ▼
[Active use: sign, encrypt, wrap, etc.]
    │
    ▼
RawKeyMaterial::drop()
    │
    ├── self.0.zeroize()                ← Overwrite all bytes with 0x00
    │
    ├── munlock_buffer(ptr, len)        ← Unlock (now-zeroed) pages
    │
    └── Vec<u8> deallocated             ← Rust allocator frees memory
```

Source: `src/store/key_material.rs:42-50`

Zeroization for other CSP types:

CSP	Zeroization Mechanism
PIN hashes	`Zeroizing<Vec<u8>>` (auto-zeroed on drop)
DRBG K, V, last_output	`#[derive(ZeroizeOnDrop)]` on struct fields
Intermediate plaintext	`Zeroizing<Vec<u8>>` in `ActiveOperation` variants
Mechanism parameters	`Zeroizing<Vec<u8>>` in `ActiveOperation` variants
Storage encryption key	Explicit `clear_persist_key()` on logout/reinit

Zeroization Verification: 7 tests in `tests/zeroization.rs` verify that key material is cleared after drop (run in debug mode, require `--ignored` flag).

8.7 Key Lifecycle (SP 800-57)

The module implements SP 800-57 key lifecycle states with date-based transitions:

Source: `src/store/object.rs` (`KeyLifecycleState` enum, `effective_lifecycle_state()`)

State	Condition	Permitted Operations
Pre-Activation	Current date < `CKA_START_DATE`	None
Active	`CKA_START_DATE` ≤ current date ≤ `CKA_END_DATE` (or no dates set)	All: sign, verify, encrypt, decrypt, wrap, unwrap, derive
Deactivated	Current date > `CKA_END_DATE`	Verify, decrypt, unwrap only
Compromised	Manually set	None
Destroyed	After `C_DestroyObject`	Handle invalid (`CKR_OBJECT_HANDLE_INVALID`)

State Transition Diagram:

```
                    ┌──────────────┐
Key Created →       │ Pre-Activation│
                    └──────┬───────┘
                           │ CKA_START_DATE reached
                    ┌──────▼───────┐
                    │    Active    │
                    └──┬──────┬──┬─┘
CKA_END_DATE        │      │  │ Manual compromise
   passed           │      │  │
              ┌─────▼──┐   │  ┌▼──────────┐
              │Deactivated│  │  │ Compromised│
              └────┬───┘   │  └────┬──────┘
                   │       │       │
        C_DestroyObject    │  C_DestroyObject
                   │       │       │
                   ┌──▼────▼───────▼──┐
                   │     Destroyed    │
                   └──────────────────┘
```

---

9. Self-Tests

9.1 Power-On Self-Tests (POST)

POST runs automatically during `C_Initialize` before any cryptographic service is available. All tests must pass; failure of any test sets `POST_FAILED: AtomicBool` (`src/pkcs11_abi/functions.rs:62`) and all subsequent operations return `CKR_GENERAL_ERROR`.

Source: `src/crypto/self_test.rs:19-57` (`run_post()`)

9.1.1 Software Integrity Test (§9.4)

Property	Value
Algorithm	HMAC-SHA256
Execution order	First (before any KATs)
Source	`integrity.rs:34` (`check_integrity()`)

See Section 4.1 for full details.

9.1.2 Known Answer Tests

#	Algorithm	Test Type	Test Vector Source	Source Function	Line
1	SHA-256	Known answer	NIST FIPS 180-4 ("abc")	`post_sha256_kat()`	64
2	SHA-384	Known answer (prefix)	NIST FIPS 180-4 ("abc")	`post_sha384_kat()`	80
3	SHA-512	Known answer (prefix)	NIST FIPS 180-4 ("abc")	`post_sha512_kat()`	94
4	SHA3-256	Known answer	NIST FIPS 202 ("abc")	`post_sha3_256_kat()`	108
5	HMAC-SHA256	Known answer	RFC 4231 Test Case 2	`post_hmac_sha256_kat()`	129
6	HMAC-SHA384	Known answer (prefix)	RFC 4231 Test Case 2	`post_hmac_sha384_kat()`	153
7	HMAC-SHA512	Known answer (prefix)	RFC 4231 Test Case 2	`post_hmac_sha512_kat()`	177
8	AES-256-GCM	Encrypt/decrypt roundtrip	Fixed key (0x42×32)	`post_aes_gcm_kat()`	206
9	AES-256-CBC	Encrypt/decrypt roundtrip	Fixed key (0x55×32), IV (0xAA×16)	`post_aes_cbc_kat()`	220
10	AES-256-CTR	Encrypt/decrypt roundtrip	Fixed key (0x77×32), IV (0xBB×16)	`post_aes_ctr_kat()`	235
11	ECDSA P-256	Sign/verify roundtrip	Generated key pair	`post_ecdsa_p256_kat()`	255
12	ML-DSA-44	Sign/verify roundtrip	Generated key pair	`post_ml_dsa_kat()`	272
13	ML-KEM-768	Encap/decap roundtrip	Generated key pair	`post_ml_kem_kat()`	285

9.1.3 RNG Health Tests

#	Test	Description	Source
14	OS RNG health	Generate 256 bytes, verify not all zeros/identical; generate second 256 bytes, verify differs from first (SP 800-90B §4.3)	`post_rng_health()` line 306
15	HMAC_DRBG health	Instantiate DRBG, generate two 32-byte outputs, verify not all zeros and consecutive outputs differ	`post_drbg_health()` line 334

9.2 Conditional Self-Tests (§9.6)

9.2.1 Pairwise Consistency Tests

Executed immediately after every asymmetric key pair generation. Source: `src/crypto/pairwise_test.rs`

Key Type	Test	Algorithm	Trigger	Source Function	Line
RSA	Sign/verify	SHA-256 RSA PKCS#1v15	`C_GenerateKeyPair(CKM_RSA_PKCS_KEY_PAIR_GEN)`	`rsa_pairwise_test()`	16
ECDSA P-256	Sign/verify	ECDSA P-256	`C_GenerateKeyPair(CKM_EC_KEY_PAIR_GEN, P-256)`	`ecdsa_p256_pairwise_test()`	54
ECDSA P-384	Sign/verify	ECDSA P-384	`C_GenerateKeyPair(CKM_EC_KEY_PAIR_GEN, P-384)`	`ecdsa_p384_pairwise_test()`	86
Ed25519	Sign/verify	Ed25519	`C_GenerateKeyPair(CKM_EDDSA)`	`ed25519_pairwise_test()`	118
ML-DSA	Sign/verify	ML-DSA-44/65/87	`C_GenerateKeyPair(CKM_ML_DSA_*)`	`ml_dsa_pairwise_test()`	150
SLH-DSA	Sign/verify	SLH-DSA-SHA2-128s/256s	`C_GenerateKeyPair(CKM_SLH_DSA_*)`	`slh_dsa_pairwise_test()`	186
ML-KEM	Encap/decap	ML-KEM-512/768/1024	`C_GenerateKeyPair(CKM_ML_KEM_*)`	`ml_kem_pairwise_test()`	221

Test data: `b"FIPS 140-3 pairwise consistency test"` (`pairwise_test.rs:13`)

9.2.2 Continuous Random Number Tests

Test	Trigger	Algorithm	Source
Continuous RNG	Every `C_GenerateRandom` call	Compare consecutive 256-byte OS RNG outputs	`functions.rs` (C_GenerateRandom)
Continuous DRBG	Every `HmacDrbg::generate()` call	Compare consecutive 32-byte DRBG outputs	`drbg.rs:42-44` (last_output comparison)

9.3 POST Failure Behavior

When any POST or conditional test fails:

1. `POST_FAILED` (`AtomicBool`) is set to `true` (`functions.rs:62`)
2. The `get_hsm()` function checks this flag on every subsequent PKCS#11 call (`functions.rs:83`)
3. All operations return `CKR_GENERAL_ERROR` (0x00000005)
4. The module cannot recover without process restart
5. No cryptographic services are available in the error state

---

10. Life-Cycle Assurance

10.1 Configuration Management

Aspect	Tool/Process
Source control	Git
Dependency pinning	`Cargo.lock` committed to repository
Dependency vetting	`cargo audit` (CVE check), `cargo deny` (license + advisory compliance)
License policy	`deny.toml` configuration
CI pipeline	GitHub Actions (build, test, lint, security audit, Miri)
Undefined behavior detection	`cargo +nightly miri test` in CI

10.2 Module Design

Central Coordinator: `HsmCore` (`src/core.rs:18-29`)

```
pub struct HsmCore {
    pub slot_manager: SlotManager,
    pub session_manager: SessionManager,
    pub object_store: ObjectStore,
    pub audit_log: AuditLog,
    pub crypto_backend: Arc<dyn CryptoBackend>,
    pub drbg: parking_lot::Mutex<HmacDrbg>,
    pub algorithm_config: AlgorithmConfig,
}
```

Backend Abstraction: `CryptoBackend` trait (`src/crypto/backend.rs`) defines 26 methods covering all cryptographic operations. Two implementations:

1. `RustCryptoBackend` (`src/crypto/rustcrypto_backend.rs`) — default, pure Rust
2. `AwsLcBackend` (`src/crypto/awslc_backend.rs`) — optional, FIPS 140-3 validated

Backend selection is configuration-driven (`algorithms.crypto_backend` in `rusthsm.toml`) and resolved at `C_Initialize` time.

10.3 Finite State Model

Module-Level States

```
┌──────────┐     C_Initialize      ┌──────────────┐
│ Power Off├──────────────────────►│ POST Running │
└──────────┘                       └──────┬───────┘
                                          │
                        ┌─────────────────┼────────────────┐
                        │ All tests pass  │                │ Any test fails
                        ▼                 │                ▼
                 ┌───────────────┐        │        ┌──────────────┐
                 │  Initialized  │        │        │  Error State │
                 │  (Ready)      │        │        │  POST_FAILED │
                 └───────┬───────┘        │        └──────────────┘
                         │                │       All ops return
                   C_Login                │       CKR_GENERAL_ERROR
                         ▼                │
                 ┌───────────────┐        │
                 │ Authenticated │        │
                 │ (CO or User)  │        │
                 └───────┬───────┘        │
                         │                │
                   C_Logout               │
                         │                │
                         ▼                │
                  Initialized ◄───────────┘
                         │
                   C_Finalize
                         ▼
                   Power Off
```

Session-Level States (PKCS#11)

```
C_OpenSession(RO)                C_OpenSession(RW)
      |                                 |
      ▼                                 ▼
┌──────────┐                     ┌──────────┐
│ RO Public │                     │ RW Public │
│ (CKS=0)   │                     │ (CKS=2)   │
└──────┬────┘                     └──┬──────┬─┘
       |                             |      |
C_Login|(User)            C_Login    |      | C_Login
       |                 (User)     |      | (SO)
       ▼                            ▼      ▼
┌──────────┐                 ┌─────────┐ ┌────────┐
│ RO User   │                 │ RW User  | | RW SO  |
│ (CKS=1)   │                 | (CKS=3)  | | (CKS=4)|
└──────┬────┘                 └────┬────┘ └───┬────┘
       |                           |          |
C_Logout                     C_Logout     C_Logout
       |                           |          |
       ▼                           ▼          ▼
 RO Public                   RW Public    RW Public
```

Source: `src/session/session.rs:7-13` (enum), lines 140-173 (transitions)

10.4 Development Environment

Aspect	Value
Language	Rust (Edition 2021)
Compiler	rustc 1.83+
Build system	Cargo
Unsafe code in crypto paths	0 blocks
Unsafe code total	Limited to FFI boundary (`src/pkcs11_abi/functions.rs`) and memory locking (`src/crypto/mlock.rs`)
Panic safety	`catch_unwind` on every `extern "C"` function

10.5 Delivery and Operation

Distribution:

- Source code via Git repository
- Compiled binaries via CI pipeline artifacts
- Release signing: GPG, Sigstore/cosign, or Windows Authenticode
- Source: `docs/release-signing.md`

Integrity Verification:

- HMAC-SHA256 sidecar file (`.hmac`) for runtime integrity test
- SHA-256 checksums for distribution verification
- Source: `tools/compute-integrity-hmac.sh`, `tools/compute-integrity-hmac.ps1`

Configuration:

- File: `rusthsm.toml` (TOML format)
- Location: `RUSTHSM_CONFIG` environment variable or working directory
- Source: `src/config/config.rs`

10.6 FIPS Build Procedures

```
# Build with FIPS-validated backend
cargo build --release --features awslc-backend

# Run full test suite
cargo test --release --features awslc-backend -- --test-threads=1

# Compute integrity HMAC
./tools/compute-integrity-hmac.sh target/release/librusthsm.so
```

FIPS configuration (`rusthsm.toml`):

```
[algorithms]
crypto_backend = "awslc"
fips_approved_only = true
enable_pqc = false
allow_sha1_signing = false
```

10.7 Guidance Documentation

Document	Path	Description
Installation guide	`docs/install.md`	Build, test, deploy, configure
Operator runbook	`docs/operator-runbook.md`	Daily operations, troubleshooting
FIPS deployment guide	`docs/fips-mode-guide.md`	FIPS mode configuration and verification
Security policy	`docs/security-policy.md`	Formal security policy (v3.0)
Architecture	`docs/architecture.md`	System design and data flow

---

11. Mitigation of Other Attacks

FIPS 140-3 Level 1 does not require mitigation of attacks beyond the module boundary. However, the module voluntarily implements the following mitigations:

Mitigation	Attack Addressed	Implementation	Source
Constant-time PIN comparison	Timing side-channel on authentication	`subtle::ConstantTimeEq` in PBKDF2 verification	`src/store/encrypted_store.rs`
RSA blinding	Timing side-channel on RSA private key	Default in `rsa` crate / aws-lc-rs	Library default
AES-NI hardware instructions	Cache-timing side-channel on AES	Hardware intrinsics when available	`aes` crate autodetection
Memory locking (mlock/VirtualLock)	Cold boot attack, swap file recovery	`RawKeyMaterial::new()`	`src/store/key_material.rs:14-18`
Memory zeroization	Memory remanence	`ZeroizeOnDrop` on all key material	`src/store/key_material.rs:42-50`
Tamper-evident audit log	Unauthorized operation detection	Chained SHA-256 hashes	`src/audit/log.rs`
Fork detection	State corruption after Unix fork	PID comparison on every call	`src/pkcs11_abi/functions.rs:67-74`
Database file locking	Concurrent access corruption	`fs2::try_lock_exclusive()`	`src/store/encrypted_store.rs:48`
Debug output redaction	Accidental key leakage in logs	Custom `Debug` impl with `[REDACTED]`	`src/store/key_material.rs:54-61`
Intermediate zeroization	Plaintext leakage in multi-part ops	`Zeroizing<Vec<u8>>` in `ActiveOperation`	`src/session/session.rs:44-47`

Appendix A: Complete Mechanism Table

All 41 cryptographic mechanisms supported by the module:

RSA Mechanisms (FIPS 186-5):

Mechanism	CKM Value	Operations	FIPS Approved	Key Size
`CKM_RSA_PKCS_KEY_PAIR_GEN`	0x00000000	Key pair generation	Yes	2048, 3072, 4096 bits
`CKM_RSA_PKCS`	0x00000001	Sign, Verify	Yes	2048+ bits
`CKM_SHA256_RSA_PKCS`	0x00000040	Sign, Verify	Yes	2048+ bits
`CKM_SHA384_RSA_PKCS`	0x00000041	Sign, Verify	Yes	2048+ bits
`CKM_SHA512_RSA_PKCS`	0x00000042	Sign, Verify	Yes	2048+ bits
`CKM_RSA_PKCS_PSS`	0x0000000D	Sign, Verify	Yes	2048+ bits
`CKM_SHA256_RSA_PKCS_PSS`	0x00000043	Sign, Verify	Yes	2048+ bits
`CKM_SHA384_RSA_PKCS_PSS`	0x00000044	Sign, Verify	Yes	2048+ bits
`CKM_SHA512_RSA_PKCS_PSS`	0x00000045	Sign, Verify	Yes	2048+ bits
`CKM_RSA_PKCS_OAEP`	0x00000009	Encrypt, Decrypt	Yes	2048+ bits

ECDSA Mechanisms (FIPS 186-5):

Mechanism	CKM Value	Operations	FIPS Approved	Curves
`CKM_EC_KEY_PAIR_GEN`	0x00001040	Key pair generation	Yes	P-256, P-384
`CKM_ECDSA`	0x00001041	Sign, Verify	Yes	P-256, P-384
`CKM_ECDSA_SHA256`	0x00001044	Sign, Verify	Yes	P-256, P-384
`CKM_ECDSA_SHA384`	0x00001045	Sign, Verify	Yes	P-256, P-384
`CKM_ECDSA_SHA512`	0x00001046	Sign, Verify	Yes	P-256, P-384

ECDH Mechanisms (SP 800-56A):

Mechanism	CKM Value	Operations	FIPS Approved	Curves
`CKM_ECDH1_DERIVE`	0x00001050	Key derivation	Yes	P-256, P-384
`CKM_ECDH1_COFACTOR_DERIVE`	0x00001051	Key derivation	Yes	P-256, P-384

EdDSA Mechanism:

Mechanism	CKM Value	Operations	FIPS Approved
`CKM_EDDSA`	0x00001057	Keygen, Sign, Verify	No

AES Mechanisms (FIPS 197, SP 800-38A/D/F):

Mechanism	CKM Value	Operations	FIPS Approved	Key Size
`CKM_AES_KEY_GEN`	0x00001080	Key generation	Yes	128, 192, 256 bits
`CKM_AES_CBC`	0x00001082	Encrypt, Decrypt	Yes	128, 192, 256 bits
`CKM_AES_CBC_PAD`	0x00001085	Encrypt, Decrypt	Yes	128, 192, 256 bits
`CKM_AES_CTR`	0x00001086	Encrypt, Decrypt	Yes	128, 192, 256 bits
`CKM_AES_GCM`	0x00001087	Encrypt, Decrypt	Yes	128, 192, 256 bits
`CKM_AES_KEY_WRAP`	0x00002109	Wrap, Unwrap	Yes	128, 192, 256 bits
`CKM_AES_KEY_WRAP_KWP`	0x0000210B	Wrap, Unwrap	Yes	128, 192, 256 bits

Digest Mechanisms (FIPS 180-4, FIPS 202):

Mechanism	CKM Value	Operations	FIPS Approved	Output Size
`CKM_SHA_1`	0x00000220	Digest	Yes (digest only)	20 bytes
`CKM_SHA256`	0x00000250	Digest	Yes	32 bytes
`CKM_SHA384`	0x00000260	Digest	Yes	48 bytes
`CKM_SHA512`	0x00000270	Digest	Yes	64 bytes
`CKM_SHA3_256`	0x000002B0	Digest	Yes	32 bytes
`CKM_SHA3_384`	0x000002C0	Digest	Yes	48 bytes
`CKM_SHA3_512`	0x000002D0	Digest	Yes	64 bytes

Post-Quantum Mechanisms (vendor-defined, not FIPS-approved):

Mechanism	CKM Value	Operations	Standard
`CKM_ML_KEM_512`	0x80000001	Keygen, Encap, Decap	FIPS 203
`CKM_ML_KEM_768`	0x80000002	Keygen, Encap, Decap	FIPS 203
`CKM_ML_KEM_1024`	0x80000003	Keygen, Encap, Decap	FIPS 203
`CKM_ML_DSA_44`	0x80000010	Keygen, Sign, Verify	FIPS 204
`CKM_ML_DSA_65`	0x80000011	Keygen, Sign, Verify	FIPS 204
`CKM_ML_DSA_87`	0x80000012	Keygen, Sign, Verify	FIPS 204
`CKM_SLH_DSA_SHA2_128S`	0x80000020	Keygen, Sign, Verify	FIPS 205
`CKM_SLH_DSA_SHA2_256S`	0x80000021	Keygen, Sign, Verify	FIPS 205
`CKM_HYBRID_ML_DSA_ECDSA`	0x80000030	Keygen, Sign, Verify	Composite

Source: `src/pkcs11_abi/constants.rs:92-138`

Appendix B: Error Code Mapping

Complete mapping from internal `HsmError` variants to PKCS#11 `CK_RV` return codes:

Source: `src/error.rs:104-157`

HsmError Variant	CK_RV Constant	Value
`NotInitialized`	`CKR_CRYPTOKI_NOT_INITIALIZED`	0x00000190
`AlreadyInitialized`	`CKR_CRYPTOKI_ALREADY_INITIALIZED`	0x00000191
`ArgumentsBad`	`CKR_ARGUMENTS_BAD`	0x00000007
`SlotIdInvalid`	`CKR_SLOT_ID_INVALID`	0x00000003
`TokenNotPresent`	`CKR_TOKEN_NOT_PRESENT`	0x000000E0
`TokenNotInitialized`	`CKR_TOKEN_NOT_RECOGNIZED`	0x000000E1
`SessionHandleInvalid`	`CKR_SESSION_HANDLE_INVALID`	0x000000B3
`SessionCount`	`CKR_SESSION_COUNT`	0x000000B1
`SessionReadOnly`	`CKR_SESSION_READ_ONLY`	0x000000B5
`SessionExists`	`CKR_SESSION_EXISTS`	0x000000B6
`SessionReadOnlyExists`	`CKR_SESSION_READ_ONLY_EXISTS`	0x000000B7
`SessionReadWriteSoExists`	`CKR_SESSION_READ_WRITE_SO_EXISTS`	0x000000B8
`UserAlreadyLoggedIn`	`CKR_USER_ALREADY_LOGGED_IN`	0x00000100

UserNotLoggedIn	CKR_USER_NOT_LOGGED_IN	0x00000101
UserTypeInvalid	CKR_USER_TYPE_INVALID	0x00000103
UserAnotherAlreadyLoggedIn	CKR_USER_ANOTHER_ALREADY_LOGGED_IN	0x00000104
UserPinNotInitialized	CKR_USER_PIN_NOT_INITIALIZED	0x00000102
PinIncorrect	CKR_PIN_INCORRECT	0x000000A0
PinInvalid	CKR_PIN_INVALID	0x000000A1
PinLenRange	CKR_PIN_LEN_RANGE	0x000000A2
PinLocked	CKR_PIN_LOCKED	0x000000A4
ObjectHandleInvalid	CKR_OBJECT_HANDLE_INVALID	0x00000082
AttributeTypeInvalid	CKR_ATTRIBUTE_TYPE_INVALID	0x00000012
AttributeValueInvalid	CKR_ATTRIBUTE_VALUE_INVALID	0x00000013
AttributeReadOnly	CKR_ATTRIBUTE_READ_ONLY	0x00000010
AttributeSensitive	CKR_ATTRIBUTE_SENSITIVE	0x00000011
TemplateIncomplete	CKR_TEMPLATE_INCOMPLETE	0x000000D0
TemplateInconsistent	CKR_TEMPLATE_INCONSISTENT	0x000000D1
MechanismInvalid	CKR_MECHANISM_INVALID	0x00000070
MechanismParamInvalid	CKR_MECHANISM_PARAM_INVALID	0x00000071
KeyHandleInvalid	CKR_KEY_HANDLE_INVALID	0x00000060
KeyTypeInconsistent	CKR_KEY_TYPE_INCONSISTENT	0x00000063
KeySizeRange	CKR_KEY_SIZE_RANGE	0x00000062
KeyFunctionNotPermitted	CKR_KEY_FUNCTION_NOT_PERMITTED	0x00000068
OperationActive	CKR_OPERATION_ACTIVE	0x00000090
OperationNotInitialized	CKR_OPERATION_NOT_INITIALIZED	0x00000091
DataInvalid	CKR_DATA_INVALID	0x00000020
DataLenRange	CKR_DATA_LEN_RANGE	0x00000021
EncryptedDataInvalid	CKR_ENCRYPTED_DATA_INVALID	0x00000040
EncryptedDataLenRange	CKR_ENCRYPTED_DATA_LEN_RANGE	0x00000041
SignatureInvalid	CKR_SIGNATURE_INVALID	0x000000C0
SignatureLenRange	CKR_SIGNATURE_LEN_RANGE	0x000000C1
BufferTooSmall	CKR_BUFFER_TOO_SMALL	0x00000150
FunctionNotSupported	CKR_FUNCTION_NOT_SUPPORTED	0x00000054
GeneralError	CKR_GENERAL_ERROR	0x00000005
HostMemory	CKR_HOST_MEMORY	0x00000002
TokenWriteProtected	CKR_TOKEN_WRITE_PROTECTED	0x000000E2
RandomSeedNotSupported	CKR_RANDOM_SEED_NOT_SUPPORTED	0x00000120

Appendix C: Dependency Inventory

Critical cryptographic dependencies inside the module boundary:

Crate	Version	Purpose	FIPS Status
`rsa`	0.9	RSA keygen, sign, verify, encrypt	Not validated (RustCrypto)
`p256`	0.13	ECDSA P-256, ECDH P-256	Not validated (RustCrypto)
`p384`	0.13	ECDSA P-384, ECDH P-384	Not validated (RustCrypto)
`ed25519-dalek`	2.2	Ed25519 sign/verify	Not validated
`aes-gcm`	0.10	AES-GCM encrypt/decrypt	Not validated (RustCrypto)
`aes`	0.8	AES block cipher	Not validated (RustCrypto)
`cbc`	0.1	AES-CBC mode	Not validated (RustCrypto)
`ctr`	0.9	AES-CTR mode	Not validated (RustCrypto)
`aes-kw`	0.2	AES Key Wrap (RFC 3394)	Not validated (RustCrypto)
`sha2`	0.10	SHA-256, SHA-384, SHA-512	Not validated (RustCrypto)
`sha3`	0.10	SHA3-256, SHA3-384, SHA3-512	Not validated (RustCrypto)
`sha1`	0.10	SHA-1 (digest only)	Not validated (RustCrypto)
`hmac`	0.12	HMAC (SHA-256/384/512)	Not validated (RustCrypto)
`pbkdf2`	0.12	PBKDF2-HMAC-SHA256	Not validated (RustCrypto)
`ml-kem`	0.3.0-rc.0	ML-KEM-512/768/1024 (FIPS 203)	Not validated
`ml-dsa`	0.1.0-rc.7	ML-DSA-44/65/87 (FIPS 204)	Not validated
`slh-dsa`	0.2.0-rc.4	SLH-DSA-SHA2-128s/256s (FIPS 205)	Not validated
`aws-lc-rs`	1.0 (optional)	FIPS-validated crypto backend	**FIPS 140-3 validated**
`zeroize`	1.8	Cryptographic memory zeroization	N/A (utility)
`subtle`	2.6	Constant-time operations	N/A (utility)
`redb`	2.6	Persistent key-value storage	N/A (storage)
`fs2`	0.4	Cross-platform file locking	N/A (utility)
`dashmap`	6.1	Lock-free concurrent hash map	N/A (utility)

Note: When built with `--features awslc-backend` and configured with `crypto_backend = "awslc"`, all cryptographic operations are performed through the FIPS 140-3 validated `aws-lc-rs` library instead of the individual RustCrypto crates.

Appendix D: Test Suite Summary

Total: **547 tests** across **29 test suites**, plus **5 fuzz targets**.

Suite	Tests	Coverage Area
Unit tests (lib)	15	POST verification, DRBG, mlock, integrity, pairwise
`crypto_vectors`	12	AES-GCM, RSA sign/verify, key sizes
`crypto_vectors_phase2`	56	ECDH, AES-CBC/CTR, RSA-OAEP/PSS, key wrap, digests, PBKDF2
`pkcs11_compliance`	1	Full PKCS#11 lifecycle
`pkcs11_compliance_extended`	1	PQC ABI, PIN enforcement, FindObjects
`pqc_phase3`	19	ML-KEM, ML-DSA, SLH-DSA, hybrid roundtrips
`concurrent_stress`	5	Thread safety under concurrent keygen/sign/encrypt
`pkcs11_error_paths`	50	Error code validation for all failure modes
`session_state_machine`	42	FSM transitions, PIN validation, lockout
`attribute_validation`	24	Sensitivity, CKA_PRIVATE, template matching
`concurrent_session_stress`	6	Multi-threaded session/object management
`zeroization`	7	Key material zeroization verification (ignored by default)
`persistence`	9	Encrypted store integrity and roundtrip
`multipart_sign_verify`	1	C_SignUpdate/Final, C_VerifyUpdate/Final
`multipart_encrypt_decrypt`	1	C_EncryptUpdate/Final, C_DecryptUpdate/Final
`supplementary_functions`	15	C_CopyObject, C_DigestKey
`fips_approved_mode`	11	FIPS approved mode policy enforcement
`pairwise_consistency`	6	Pairwise consistency on all keygen paths
`pkcs11_info_functions`	25	C_GetInfo, C_GetSlotInfo, C_GetTokenInfo, C_GetMechanismInfo
`key_lifecycle_abi`	25	SP 800-57 date-based activation/deactivation
`key_wrapping_abi`	22	C_WrapKey/C_UnwrapKey roundtrips and error paths
`key_derivation_abi`	19	ECDH P-256/P-384 key derivation
`rsa_abi_comprehensive`	28	RSA 2048/3072 sign/verify/encrypt/decrypt
`digest_abi`	25	All 7 hash algorithms through C ABI
`attribute_management`	25	C_GetAttributeValue, C_SetAttributeValue, C_FindObjects
`random_and_session`	22	C_GenerateRandom, session management
`pqc_abi_comprehensive`	28	ML-DSA/ML-KEM/SLH-DSA through C ABI
`audit_and_integrity`	24	AuditLog chain integrity, StoredObject lifecycle
`negative_edge_cases`	30	Cross-algo failures, boundary conditions

Fuzz Targets (cargo-fuzz):

Target	Focus
`fuzz_c_abi`	Random PKCS#11 function calls with fuzzed parameters
`fuzz_crypto_ops`	Fuzzed key sizes, data lengths, mechanism parameters
`fuzz_attributes`	Fuzzed template construction and attribute values
`fuzz_session_lifecycle`	State machine edge cases, login/logout sequences
`fuzz_buffer_overflow`	Integer overflow, two-call pattern, null pointers

Appendix E: Configuration Reference

Complete `rusthsm.toml` schema with FIPS implications:

```
[token]
label = "RustHSM Token"          # Token label (32 bytes max, ASCII)
storage_path = ""                 # Path for persistent storage (empty = in-memory only)
max_sessions = 100                # Maximum concurrent sessions
max_rw_sessions = 50              # Maximum read-write sessions

[security]
pin_min_length = 4                # Minimum PIN length (bytes)
pin_max_length = 64               # Maximum PIN length (bytes)
max_failed_logins = 10            # Failed login attempts before lockout
pbkdf2_iterations = 600000        # PBKDF2 iteration count

[algorithms]
crypto_backend = "rustcrypto"     # "rustcrypto" (default) or "awslc" (FIPS)
fips_approved_only = false         # true = block non-FIPS mechanisms
enable_pqc = true                  # true = enable PQC mechanisms
allow_weak_rsa = false             # true = allow RSA < 2048 bits
allow_sha1_signing = false         # true = allow SHA-1 in signing contexts

[audit]
enabled = true                     # Enable audit logging
log_path = ""                      # Audit log file path (empty = in-memory only)
log_level = "all"                  # "all", "security", or "errors"

[daemon]
bind = "127.0.0.1:5696"           # gRPC bind address (daemon only)
tls_cert = ""                      # TLS certificate path
tls_key = ""                       # TLS private key path
```

FIPS Mode Configuration Requirements:

Parameter	Required Value	Reason
`crypto_backend`	`"awslc"`	Must use FIPS-validated backend
`fips_approved_only`	`true`	Block non-approved mechanisms
`enable_pqc`	`false`	PQC not yet FIPS-approved for CMVP
`allow_weak_rsa`	`false`	RSA < 2048 not approved
`allow_sha1_signing`	`false`	SHA-1 deprecated for signing (SP 800-131A)

End of FIPS 140-3 Security Policy and Certification Document RustHSM v0.8.0 — 2026-03-08